What is Programming

- Programming is the process of creating a set of instructions that a computer can follow to perform specific tasks. These instructions are written in a programming language, which provides the syntax and structure needed to create executable code.

Compiler Types

- AOT (AHead OF Time)
- JIT (JUST IN TIME)

What is Syntax in Programming Languages

syntax is the set of rules and structure that defines how code must be written to be correctly interpreted and executed by a computer

Introduction to Dart Language

Your Gateway to Modern App Development

What is Dart?

Dart is an open-source, client-optimized programming language developed by Google.

1

Easy to Learn

Familiar syntax, ideal for beginners and experienced developers.

2

Fast & Efficient

Compiles to native code for high performance.

3

Object-Oriented

Supports classes, inheritance, and other OOP principles.

Why Use Dart?

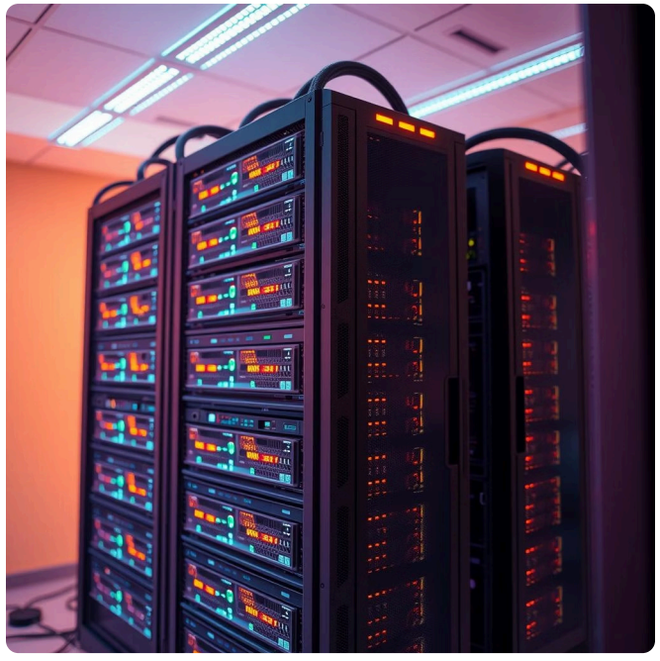
Dart powers diverse applications, from mobile to server-side.

Flutter Development

Primary language for building beautiful, fast, native cross-platform apps (iOS, Android, Web, Desktop) from a single codebase.

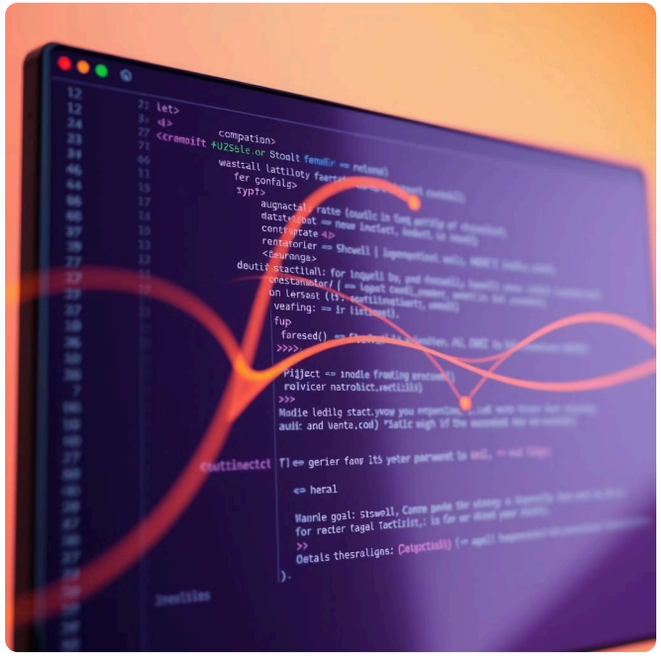
Backend Services

Build robust and scalable **server-side applications** and APIs using frameworks like Shelf.



Command-Line Tools

Create **efficient command-line interface (CLI) applications** for automation and utility tasks.



Dart Basics: Variables & Data Types

Variable Keywords

- **var:** Type inferred, value can change.
- **final:** Assigned once, cannot change later.
- **const:** Compile-time constant, fixed value.

Fundamental building blocks for storing and manipulating data.

Common Data Types

int	Whole numbers , `int age = 25;`
double	Decimal numbers , `double price = 10.5;`
String	Text , `String name = "Rana";`
bool	True/False , `bool isActive = true;`

Collections: Lists & Maps

Organize multiple data points efficiently.

Lists

Ordered, index-based collections.

- `List fruits = ['Apple', 'Banana'];`
- `fruits.add('Orange');`
- `fruits.remove('Banana');`
- `print(fruits[0]);`
- `print(fruits.length);`

Maps

Unordered, key-value pairs; unique keys.

- `Map<String, int> scores = {'Math': 90};`
- `print(scores['Math']);`
- `scores['History'] = 88;`
- `scores.remove('English');`

Null Safety in Dart

Null safety in Dart is a feature that helps developers avoid null reference errors by ensuring that variables cannot contain null values unless explicitly allowed

Benefits of Null Safety

- **Fewer Runtime Errors**

By catching null-related errors at compile time, null safety reduces the risk of runtime exceptions.

- **Improved Code Quality**

Code becomes clearer about which variables can be null, leading to better documentation and understanding of the codebase.

- **Enhanced Developer Experience**

IDEs and static analysis tools can provide better insights and warnings about potential null dereferences, improving overall developer productivity.

Null Safety: Key Concepts

Nullable vs. Non-Nullable

?

Use `?` for nullable types (`String? middleName;`) or declare non-nullable by default (`String name = "Rana";`).

Null Assertion Operator

!

Tells Dart a nullable variable is non-null (`print(name!.length);`). Use with caution, as it throws an error if null at runtime.

Late Initialization

The `late` keyword declares non-nullable variables that will be initialized later, guaranteed before use.

Control Flow: Conditionals

Guide your program's execution based on conditions.

If, Else If, Else

Execute code blocks based on boolean conditions.

```
int number = 10;
if (number > 0) {
  print("Positive");
} else if (number == 0) {
  print("Zero");
} else {
  print("Negative");
}
```

Switch-Case

Simplify checking a variable against multiple fixed values.

```
int day = 3;
switch (day) {
 case 1: print("Monday"); break;
 case 2: print("Tuesday"); break;
 default: print("Invalid");
}
```

Control Flow: Loops

Automate repetitive tasks in your code.

For Loop

Repeats a block a specific number of times.

```
for (int i = 1; i <= 5; i++) { print(i); }
```

While Loop

Repeats while a condition remains true.

```
int i = 1; while (i <= 5) { print(i); i++; }
```

Do-While Loop

Runs at least once, then repeats while a condition is true.

```
int i = 1; do { print(i); i++; } while (i <= 5);
```

Functions: Reusable Code Blocks

Organize your code for clarity and efficiency.

Define & Call

Create a task-specific code block. `void greet() { print("Hello!"); }` Call it with `greet();`

With Parameters

Pass inputs for dynamic behavior. `void greetUser(String name) { print("Hello, $name!"); }`

Return Values

Functions can return results. `int square(int num) { return num * num; }`

Optional & Named

Flexible parameter passing. `greet([String name])` or `greet({String? name})`